

[CS230] Special Applications: Face Recognition & One-shot Learning

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-9. CNN Foundations & Architectures - *Completed*

Chapter 10. Special Applications (Current Unit)

- 10.1 Face Recognition
 - * The One-shot Learning Problem
 - * Siamese Network Architecture
 - * Triplet Loss Function
 - * Binary Classification Alternative
- 10.2 Neural Style Transfer - *Upcoming*

지난 시간 복습 및 연결

지난 시간까지 우리는 YOLO를 통해 객체의 위치를 찾는 법을 배웠습니다. 오늘은 그보다 더 까다로운 문제인 "이 사람이 누구인가?"를 구별하는 얼굴 인식에 도전합니다. 우리는 스마트폰을 살 때 얼굴을 한 번만 등록합니다. 그런데 딥러닝은 보통 수천 장의 데이터가 필요합니다. 어떻게 단 한 장의 사진만으로 주인을 알아볼까요? 기존 상식을 깨는 **One-shot Learning**과 **Siamese Network**의 비밀을 파헤쳐 봅시다.

1 Unit Overview

□ 핵심 목표

이 단원은 데이터가 극도로 적은 상황에서 작동하는 얼굴 인식 시스템의 원리를 다룹니다.

- **One-shot Learning:** 단 한 장의 데이터로 학습하는 문제를 '분류'가 아닌 '유사도 측정'으로 풉니다.
- **Siamese Network:** 두 이미지를 같은 네트워크에 통과시켜 거리(Distance)를 계산하는 구조를 배웁니다.
- **Triplet Loss:** Anchor, Positive, Negative 세 장의 사진을 이용한 학습 방법을 수학적으로 유도합니다.

2 Essential Terminology

용어	의미	핵심 역할
One-shot Learning	한 번만 보고 배우기	데이터가 적은 문제 해결.
Siamese Network	쌍 네트워크	두 입력이 같은 가중치(W)를 공유함.
Encoding	$f(x)$	이미지를 128차원 등의 숫자 벡터로 변환.
Triplet Loss	세 쌍 손실 함수	A-P는 가깝게, A-N은 멀게 만듦.

3 Core Concepts: 분류가 아니라 비교다

3.1 1. The Challenge (One-shot Learning)

직원 1,000명의 출입 통제 시스템을 만든다고 가정합니다.

- **Softmax (실패):** 1,000개 클래스로 분류. 신입 사원이 오면 네트워크 전체를 재학습해야 합니다. (확장성 0점)
- **Similarity (성공):** 두 사진을 비교하여 거리(Distance) d 를 출력합니다.
 - $d(\text{img1}, \text{img2}) \leq \tau$: 같은 사람 (문 열림)
 - $d(\text{img1}, \text{img2}) > \tau$: 다른 사람 (거부)

신입 사원이 오면 사진만 DB에 추가하면 됩니다. 재학습이 필요 없습니다.

3.2 2. Siamese Network (쌍 네트워크)

[Image of Siamese network architecture with two shared CNNs feeding into distance calculation]
두 개의 똑같은 네트워크가 머리(가중치)를 공유합니다.

1. 두 이미지 $x^{(1)}, x^{(2)}$ 를 각각 CNN에 넣습니다.
2. 마지막 FC 층에서 나온 벡터(인코딩) $f(x^{(1)}), f(x^{(2)})$ 를 얻습니다.
3. 두 벡터 사이의 유클리드 거리를 계산합니다.

$$d(x^{(1)}, x^{(2)}) = \|f(x^{(1)}) - f(x^{(2)})\|^2$$

4 Deep Dive: Triplet Loss (트리플렛 손실)

쌍 네트워크를 어떻게 학습시킬까요? 세 장의 사진을 한 세트(Triplet)로 묶어 학습합니다.

- **Anchor (A):** 기준이 되는 내 사진.
- **Positive (P):** 나와 같은 사람의 다른 사진.
- **Negative (N):** 나와 다른 사람(영희)의 사진.

□ Loss Function Derivation (수학적 증명)

우리의 목표는 다음과 같습니다.

$$\|f(A) - f(P)\|^2 \leq \|f(A) - f(N)\|^2$$

(A와 P 사이의 거리가 A와 N 사이의 거리보다 작아야 한다.)

하지만 신경망이 $f(x) = 0$ (모든 출력을 0으로)으로 학습해버리면, $0 \leq 0$ 이 되어버립니다 (Trivial Solution). 이를 막기 위해 마진(α)을 둡니다.

$$\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha \leq 0$$

최종 손실 함수 (ReLU 형태):

$$L(A, P, N) = \max(0, \|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha)$$

5 Implementation: Triplet Loss

TensorFlow/Keras 스타일의 손실 함수 구현입니다.

```
1 import tensorflow as tf
2
3 def triplet_loss(y_true, y_pred, alpha=0.2):
4     """
5     y_pred: [Anchor, Positive, Negative] 임베딩이 연결된 텐서
6     """
7     # 1. 임베딩 벡터 분리 전체 (길이의 씩 1/3)
8     total_len = y_pred.shape[1]
9     emb_size = total_len // 3
10
11     anchor = y_pred[:, 0:emb_size]
12     positive = y_pred[:, emb_size:2*emb_size]
13     negative = y_pred[:, 2*emb_size:]
14
15     # 2. 거리 계산 제공함()
16     # axis=-1: 벡터의 각 성분별 합
17     pos_dist = tf.reduce_sum(tf.square(anchor - positive), axis=-1)
18     neg_dist = tf.reduce_sum(tf.square(anchor - negative), axis=-1)
19
20     # 3. Loss 계산 (Margin 추가)
21     basic_loss = pos_dist - neg_dist + alpha
22
23     # 4. max(0, loss)
24     loss = tf.maximum(basic_loss, 0.0)
25
26     return tf.reduce_mean(loss)
```

Listing 1: Triplet Loss Function

6 FAQ & Pitfalls

Hard Triplet Mining (학습 데이터 선정) (오해 방지 가이드)

랜덤하게 A, P, N을 고르면 학습이 잘 안 됩니다. 왜냐하면 대부분의 경우 랜덤한 두 사람 (A, N)은 이미 충분히 다르게 생겼기 때문입니다. ($Loss = 0$) 학습 효율을 위해 "A와 N이 꽤 닮은 경우(Hard Triplet)"를 골라내어 학습시켜야 합니다.

Q. 얼굴 말고 다른 거에도 쓸 수 있나요?

A. 네! 서명 인식, 지문 인식, 심지어 이미지 검색(쇼핑몰에서 비슷한 옷 찾기) 등 "유사한 것을 찾는" 모든 분야에 쓰입니다.

다음 단계 (Next Step)

얼굴 인식은 CNN이 추출한 '내용(Content)'을 비교하는 기술이었습니다. 그렇다면 CNN이 추출한 '화풍(Style)'만 따로 떼어낼 수도 있을까요?

다음 시간에는 반 고흐의 화풍을 내 사진에 입히는 예술적인 AI, [Neural Style Transfer]에 대해 알아보겠습니다. CNN의 깊은 층이 무엇을 보고 있는지 시각적으로 확인할 수 있는 흥미로운 시간이 될 것입니다.

□ 단원 요약 (Cheat Sheet)

1. **Similarity**: 분류가 아닌 거리 측정 문제로 접근한다.
2. **Siamese Network**: 가중치를 공유하는 쌍둥이 네트워크.
3. **Triplet Loss**: (A, P, N) 구조. A-P는 당기고, A-N은 민다.
4. **Margin α** : 모델이 모든 출력을 0으로 만드는 꼼수를 방지한다.